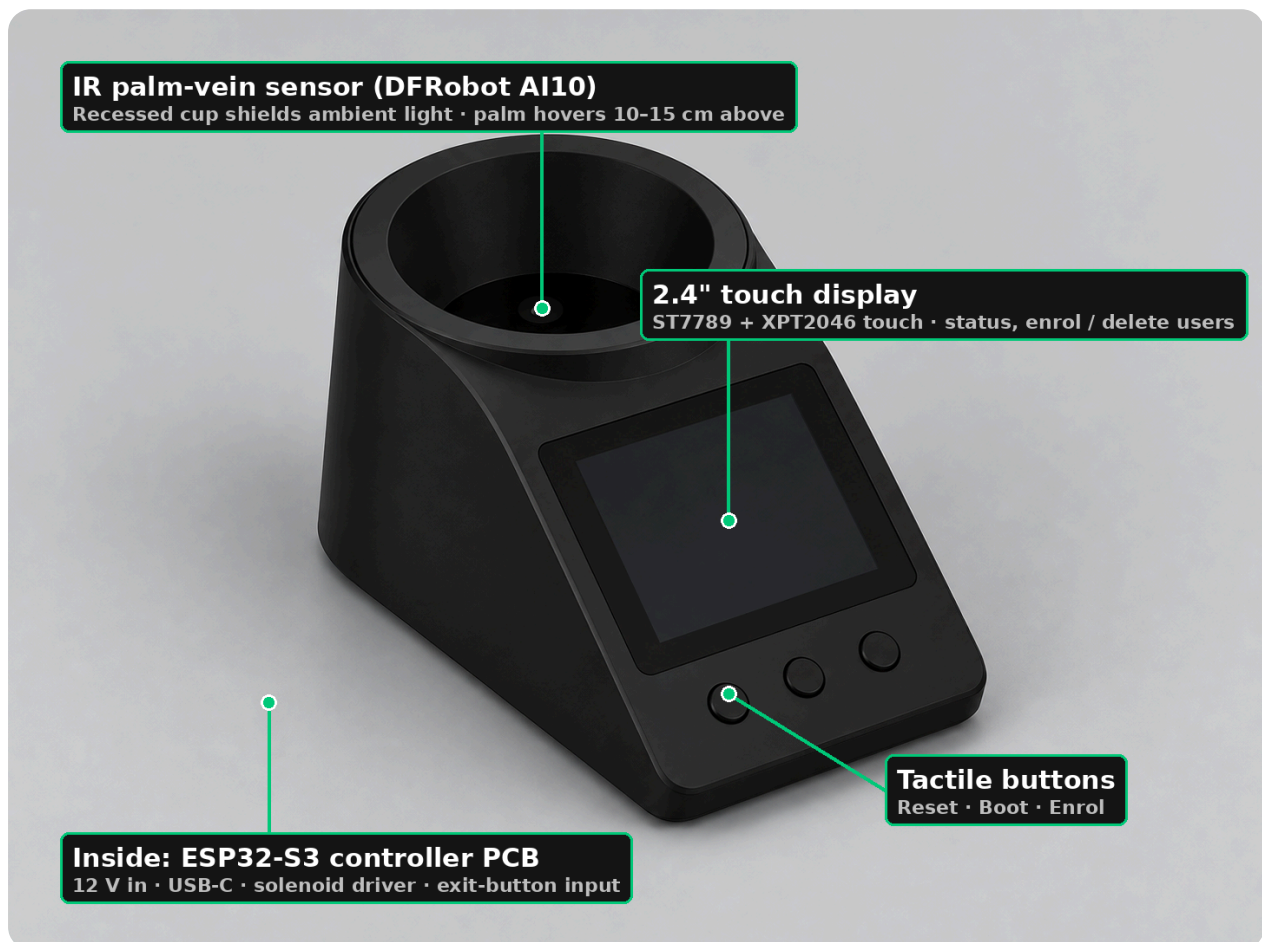


palm-scan

Wiring & Components Guide — every connection, explained for everyone



APPLIES TO

Controller board Rev 2.4 (schematic verified 14 Sep 2026)
DFRobot AI10 palm-vein sensor · 2.4" TFT touch display
· 12 V solenoid

MAKERS

Ashmita K Rao · Adithya Satish · Vinod Kumar
github.com/coolvnod/palm-scan

Contents

1 How to read this guide	3
2 The system at a glance	5
3 The complete wiring diagram (A3 fold-out)	6
4 Connection by connection	7
4.1 Power in — the wall adapter	7
4.2 The palm-vein sensor	8
4.3 The touch screen	9
4.4 The solenoid bolt	11
4.5 The exit button	12
4.6 The front-panel lights	13
4.7 USB-C — programming and logs	14
4.8 The debug header (normally empty)	15
5 Inside the board — the parts and what they do	16
6 Where the power goes	18
7 What happens during a scan	19
8 Before you power it on	20
A Appendix — pin tables	21
B Appendix — glossary	23

Section 3 is an A3 fold-out. If you print on A4, print that page at “fit to page” — or read it on a screen, where you can zoom.

1 How to read this guide

This guide shows every wire and every part of the palm-scan controller, and says what each one is for — in ordinary words first, and in engineering terms for those who want them.

You do not need to know electronics. Three things are enough: a wire carries either *power* or *information*; power has a + side and a – side; and every connector has numbered *pins*. Everything else is explained where it comes up.

The guide has three layers, marked like this:

IN PLAIN WORDS

Ordinary-language explanation. If you only read these, you can still wire the unit correctly.

FOR THE ENGINEER

Precise engineering detail — part numbers, voltages, currents, and the reasoning behind a choice. Skip freely.

BEFORE YOU CONNECT IT

Something to check or avoid *before* plugging in. These are the paragraphs that save hardware.

Wire colours used in the drawings

Colours in this guide follow what you will actually see: the sensor cable comes with its wires already coloured by the maker, and the rest follow the usual convention of red for +, black for –.

Colour	Meaning	Where you will see it
 red	12 V power, the positive (+) side	adapter → board, board → lock
 orange	5 V power	regulator → sensor (inside the board)
 yellow-gold	3.3 V power	board → screen, debug header
 black	ground — the shared return path, the “–” side	every cable has one
 yellow	data: the board talking to the sensor	sensor cable
 green	data: the sensor talking to the board	sensor cable
 white	a switched line — the board turns it on or off	lock, exit button
 purple	USB	laptop cable
 grey	a sheath with several wires inside	sensor cable

Ten words

Word	What it means here
Voltage (V)	How hard electricity is being pushed — like water pressure. 12 V pushes harder than 3.3 V. Every part has a voltage it expects; too much breaks it.
Current (A)	How much electricity is flowing — like litres per minute. The adapter can supply 2 A; the board never needs more than about 1.2 A.
Ground (GND, -)	The common “zero level” every voltage is measured against, and the path electricity takes back. Every cable carries one ground wire.
Rail	A shared supply line at one voltage. The board has three: +12 V, +5 V and +3.3 V.
Signal	A wire that carries information instead of power. Tiny currents, but the timing matters.
Pin	One metal contact in a connector. Pins are numbered; pin 1 is usually marked with a square, a dot or an arrow.
Polarity	Which way round. Power and LEDs have a + and a -; a button and the solenoid don't care.
UART	A two-wire conversation: one wire for each direction, at an agreed speed. The sensor talks to the board this way.
SPI	A faster, four-wire conversation with a shared clock — used for the screen.
GPIO	A pin on the brain that the software can switch on or off, or read. The lock, buzzer, LEDs and button all sit on GPIOs.

2 The system at a glance

One small board is the brain. Eight things connect to it. That is the whole system.

The board is built around an **ESP32-S3** — a microcontroller with Wi-Fi, Bluetooth and a USB port on a thumbnail-sized module. It does not do the palm recognition itself: the **AI10 sensor** is a small computer of its own that photographs the palm in near-infrared light and answers the question “who is this?”. The board asks that question, reads the answer, and then does the ordinary things: lights, a beep, a message on the screen, and — if the answer is a known person — power to the bolt for a few seconds.

Everything that plugs into the board is listed below with the number used for it throughout this guide. The numbers appear on the fold-out diagram in Section 3 and as the sub-section numbers in Section 4.

What	What it does	Where it connects
1 Wall adapter	Supplies all the power: 12 V, up to 2 A.	J8 (round plug) or J1 (bare wires)
2 Palm-vein sensor	The eye. Shines near-infrared light, photographs the veins, decides who it is.	CN1 — 4-wire cable, supplied with the sensor
3 Touch screen	Shows what is happening; used to add or remove people.	J4 — plugs straight onto the board, no cable
4 Solenoid bolt	The hand. Pulls a bolt back for five seconds when a palm is recognised.	J3 — 2 wires
5 Exit button	Opens the bolt from inside without a scan.	J5 — 2 wires
6 Panel lights	Green = recognised, red = not recognised.	J7 — 3 wires
7 Laptop (USB-C)	Only for loading software and reading logs.	J2 — ordinary USB-C cable
8 Debug header	A spare way in if USB ever breaks. Normally empty.	J6 — 4 pins

IN PLAIN WORDS

Think of it as a small office: the **brain** (ESP32-S3) sits at the desk. The **eye** (sensor) reports who is at the door. The **screen**, **lights** and **beeper** talk to the person. The **bolt** is the only thing that moves. The **adapter** keeps the lights on. The **USB port** is the back door for the person who maintains it.

FOR THE ENGINEER

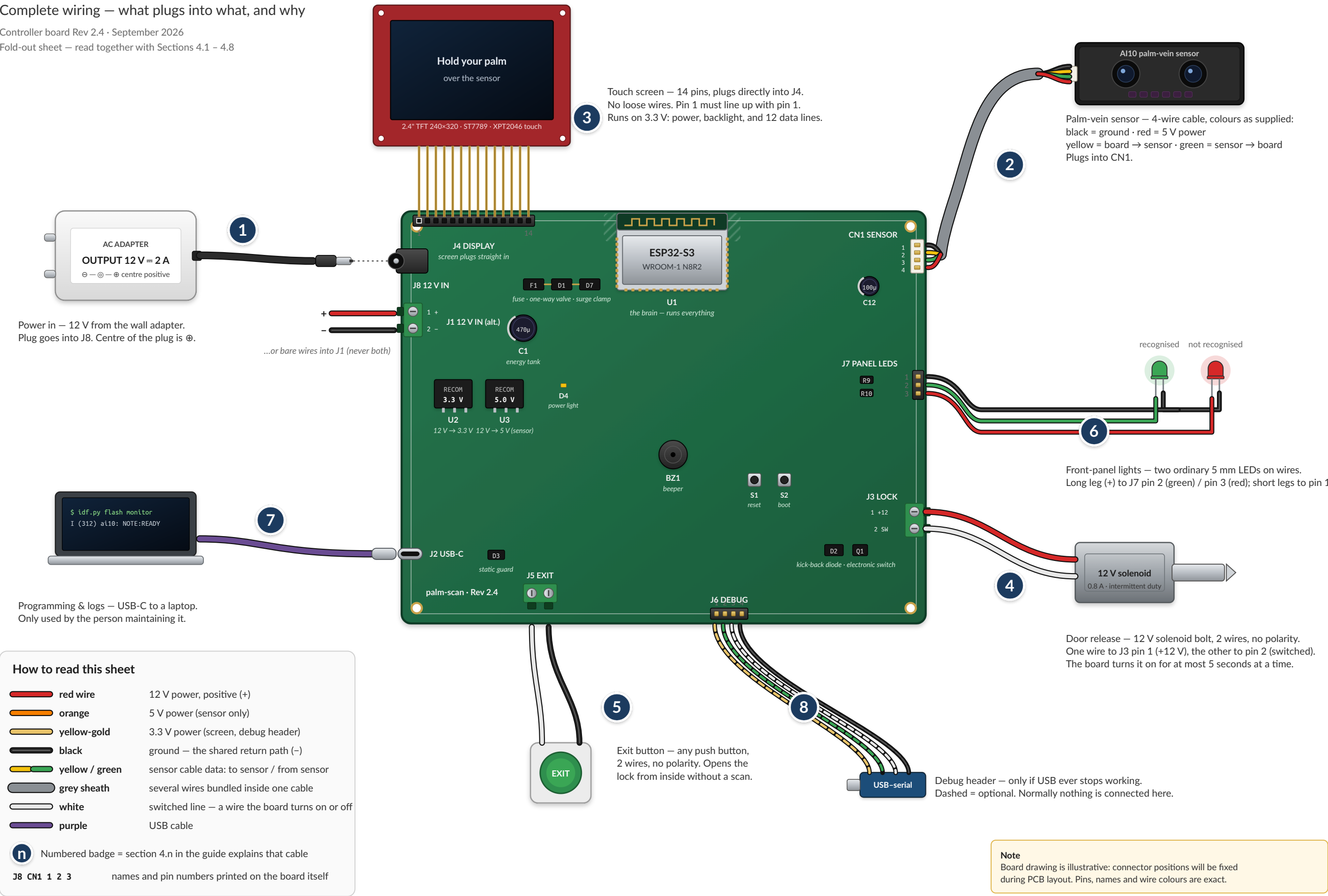
Recognition runs entirely inside the AI10 (DFRobot SEN0677): NIR illumination, binocular camera, on-module matching, template storage. The ESP32-S3 drives it over UART at 115 200 baud with a simple framed protocol (EF AA · id · length · payload · XOR). The board itself has 46 parts on 38 nets; its jobs are power conditioning (12 V in → 3.3 V and 5 V), a protected low-side MOSFET driver for the solenoid, an SPI display, USB for programming, and I/O for a button, two LEDs and a piezo. Full design record: docs/DESIGN_REVIEW.md .

palm-scan

Complete wiring — what plugs into what, and why

Controller board Rev 2.4 · September 2026

Fold-out sheet — read together with Sections 4.1 – 4.8



4 Connection by connection

Each of the eight connections, one at a time: a drawing, the pin table, why it is wired that way, and what goes wrong if it isn't. Net names (`this typeface`) are the schematic's labels.

4.1 Power in – the wall adapter

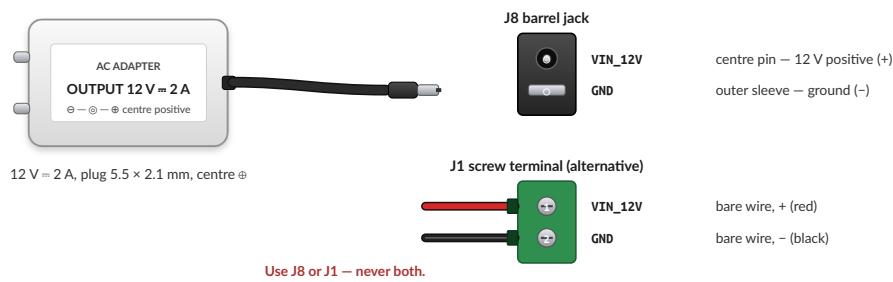


Figure 4.1 — The adapter's round plug goes into J8. The screw terminal J1 is an alternative for an adapter with bare wire ends, or a bench supply. They are wired in parallel on the board: use one or the other, never both.

Connector	Pin	Net	Plain words
J8 barrel jack	centre	VIN_12V	12 V, positive (+). The adapter must be <i>centre-positive</i> (⊕ symbol on its label).
J8 barrel jack	sleeve	GND	ground (-)
J1 screw terminal	1	VIN_12V	12 V, positive (+) – red wire
J1 screw terminal	2	GND	ground (-) – black wire

IN PLAIN WORDS

The adapter is the only source of power. Twelve volts is the “pressure” the solenoid needs; the board makes its own lower voltages from it (Section 6). Two amps is more than the unit ever draws — about 1.2 A while the bolt is pulled, a quarter of that the rest of the time. A reversed plug does no harm: a one-way valve (D1) on the board simply blocks it.

FOR THE ENGINEER

12 V DC regulated (±5 %), ≥ 1.5 A, 5.5 × 2.1 mm centre-positive. Path: J8/J1 → F1 (Bourns MF-MSMF200/16 polyfuse, 2 A hold / 4 A trip) → D1 (SS34 Schottky, 3 A, ~0.5 V) → +12V rail with C1 470 μF bulk and D7 SMAJ15A TVS (15 V stand-off, ~24 V clamp, below Q1's 30 V). Regulators need ≥ 8 V in, so 9 V is out; 19 V drives D7 into conduction and trips F1. Worst 1.5 A, idle ≈ 0.25 A (POWER_BUDGET §1).

BEFORE YOU CONNECT IT

- Label must say **12 V** and **⊕ on the centre**. Some adapters (guitar pedals, older modems) are centre-negative.
- Plug size 5.5 × 2.1 mm — a 2.5 mm plug fits loosely and flickers.
- **J8 or J1**, never both. Measure the adapter first: 11.5 – 12.6 V is right.

4.2 The palm-vein sensor

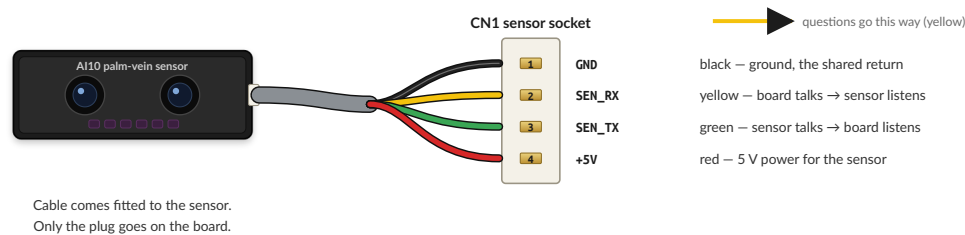


Figure 4.2 — The AI10 comes with its 4-wire cable already fitted. Only the small plug at the free end goes onto the board, into CN1. Yellow and green are the two directions of the conversation; red and black are power.

Pin	Wire colour	Net	Plain words
1	 black	GND	ground — the return path for both power and data
2	 yellow	SEN_RX	the sensor's <i>listening</i> wire: the board's questions travel along it ("reset", "who is this?", "add this person")
3	 green	SEN_TX	the sensor's <i>talking</i> wire: its answers come back along it ("ready", "user 3, score 92", "nobody")
4	 red	+5V	5 V power for the sensor, made on the board

IN PLAIN WORDS

The sensor and the board talk the way two people talk on a phone: one wire carries each direction, and both must agree on a speed (here 115 200 “bits per second”, roughly eleven thousand letters a second). The maker names the wires from the *sensor’s* point of view — its RX (receive) wire is where the board’s messages arrive — so the yellow wire is `SEN_RX` even though it carries what the board *sends*. The cable does the crossing-over for you; you only plug it in.

FOR THE ENGINEER

UART1 on the ESP32-S3: IO17 (U1TXD) → CN1 pin 2, IO18 (U1RXD) ← CN1 pin 3. 115 200 8N1, fixed. The sensor is powered from the dedicated +5V rail (U3, R-78E5.0-1.0, 1 A) rather than 12 V: the AI10 accepts 5–12 V, but at 12 V its internal regulator runs hot. Measured draw 320–330 mA at 8 V; budgeted 530 mA worst case at 5 V. C12 (100 μF) sits right at CN1 to feed the unknown start-up spike. After power-up the module sends `NOTE:READY`; firmware must wait for it before sending any command. Protocol reference: [docs/datasheets/SEN0677_AI10_protocol_manual_V1.0.pdf](#).

BEFORE YOU CONNECT IT

- **The sensor’s data voltage is not documented.** Before the plug ever touches the board: power the sensor on its own (red = 5 V, black = ground) and measure the **green** wire with a multimeter. Idle, it must read **3.3 V or less (never above 3.6 V)**. If it reads 5 V, stop — the board needs a small divider added on that wire.
- Check the plug pitch: CN1 is a 1.25 mm PicoBlade socket. If the sensor’s plug is 1.5 mm or 2.0 mm, the socket on the board must be changed before ordering the PCB.
- Pin 1 (black) is marked on the board. The plug only fits one way.

4.3 The touch screen



Figure 4.3 – The screen module has a row of 14 pins that pushes straight into the socket J4 on the board's top edge. Nothing to solder, no cable. Seven pins drive the picture; six drive the touch layer; pin 9 is deliberately left empty.

Pin	Screen label	Net	Plain words
1	VCC	+3V3	power for the screen
2	GND	GND	ground
3	CS	TFT_CS	"I'm talking to you, screen" – the board pulls this low to address the display
4	RESET	TFT_RST	restarts the screen chip at power-up
5	DC	TFT_DC	tells the screen whether the next byte is a command or picture data
6	SDI (MOSI)	SPI_MOSI	picture data, board → screen
7	SCK	SPI_SCK	the clock – a steady tick that keeps sender and receiver in step
8	LED	+3V3	backlight power
9	SDO (MISO)	–	not connected, on purpose (see engineer's note)
10	T_CLK	SPI_SCK	the same clock, shared with the touch chip
11	T_CS	TOUCH_CS	"I'm talking to you, touch chip"
12	T_DIN	SPI_MOSI	commands, board → touch chip
13	T_DO	SPI_MISO	the touch position, touch chip → board
14	T_IRQ	TOUCH_IRQ	"someone is touching me" – lets the board react instantly

IN PLAIN WORDS

Where the sensor uses two wires, the screen uses a faster arrangement called SPI: a clock wire ticks, and on every tick one bit moves along the data wire. Two chips live on the screen module — one that draws pixels and one that senses your finger — and they share the clock and data wires like two people sharing one phone line. The two CS (“chip select”) wires are how the board says which of them it is addressing. Because the board only ever *sends* to the picture chip, that chip’s reply wire (pin 9) is left unconnected.

FOR THE ENGINEER

FSPI on IOMUX pins for full speed: IO12 SCK, IO11 MOSI, IO13 MISO, IO10 TFT_CS (FSPICS0), IO9 TOUCH_CS, IO14 DC, IO21 RST, IO8 TOUCH_IRQ. The whole module runs from +3V3 (VCC and LED) so no module output can exceed the ESP32’s 3.3 V I/O — Rev 2.4 moved it off 5 V for exactly that reason; the backlight is marginally dimmer. Pin 9 is unconnected because the ST7789’s SDO does *not* tri-state when CS is high on these red 2.4” boards, so it would fight the XPT2046 on MISO — a well-known failure (touch reads garbage, or the display freezes after the first touch). The Robu listing says “ILI9341/ST7789”: firmware must probe and support both. TOUCH_IRQ is optional; polling works.

BEFORE YOU CONNECT IT

- **Pin 1 to pin 1.** The board’s socket has a square pad at pin 1 and the module’s header has pin 1 marked. Plugging in shifted by one pin puts 3.3 V on a data pin — nothing burns, but nothing works either.
- The manual lists pin *names* but not their order. When the module arrives, read the order off its silkscreen and compare with the table above before the PCB is ordered.

4.4 The solenoid bolt

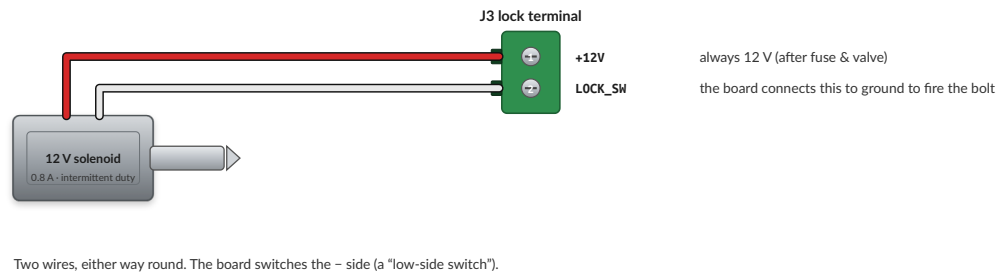


Figure 4.4 — Two wires from the solenoid into the screw terminal J3. There is no right or wrong way round for the solenoid itself: it is a coil, and the board switches its - side.

Pin	Net	Plain words
1	+12V	always at 12 V — one end of the coil sits here permanently
2	LOCK_SW	the switched end. The board connects it to ground for up to 5 s; current flows; the bolt pulls back

IN PLAIN WORDS

A solenoid is an electromagnet with a spring-loaded bolt inside. Give it 12 V and the magnet pulls the bolt in; take the power away and the spring pushes it out. The board does the “give it 12 V” with an electronic switch (a MOSFET, Q1) on the ground side of the coil, so the wiring is just two wires and a screw terminal. The board also makes sure the bolt is never held in for more than five seconds at a time — these cabinet solenoids overheat if left on.

FOR THE ENGINEER

Low-side switch: Q1 AO3400A (30 V, 5.7 A, $R_{DS(on)} \approx 40 \text{ m}\Omega$) gated from IO4 via R13 100 Ω , with R2 10 k Ω gate-to-source holding it off while the ESP32 boots (GPIOs float for ~50 ms at reset). D2 SS54 across the coil catches the inductive kick when Q1 opens. The coil is fed from +12V *after* F1 and D1 — Rev 2.0 fed it before D1, which put Q1’s body diode straight across a reversed adapter. Rated load: Robu 12 V cabinet lock, ~0.8 A, intermittent duty (max 10 s on). Firmware: hard 5 s cap per release plus the ESP-IDF task watchdog. Fail-secure: loss of power = bolt out = locked.

BEFORE YOU CONNECT IT

- Buy a solenoid marked **12 V**. Measure its current once with the multimeter: if it is above 0.8 A, tell whoever orders the PCB — the fuse must move up one size.
- If this bolt ever secures a real door, the door **must open from inside with a handle** regardless of the electronics. That is a fire-safety rule, not a preference.
- Keep this cable away from the exit-button cable and the sensor cable — the coil’s switching can upset them.

4.5 The exit button

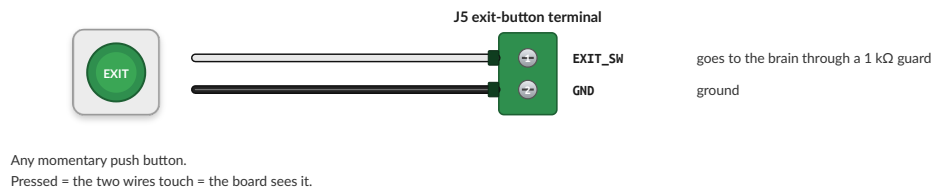


Figure 4.5 — Any ordinary push button, two wires, into J5. Either way round.

Pin	Net	Plain words
1	EXIT_SW	the sensing wire — normally held at 3.3 V by the board
2	GND	ground

IN PLAIN WORDS

A button is the simplest input there is: two wires that touch when you press. The board keeps the sensing wire gently pulled up to 3.3 V; pressing the button connects it to ground, the voltage drops to zero, and the software sees “pressed”. Real buttons bounce — the contacts chatter for a few thousandths of a second — so the board smooths the wire with a small capacitor and the software double-checks before acting.

FOR THE ENGINEER

IO6 with R6 10 kΩ pull-up to +3V3 , C11 100 nF to ground ($RC \approx 1$ ms debounce), and R8 1 kΩ in series between IO6 and the terminal to limit current from static or induced transients on a long wall wire; the ESP32’s internal clamp diodes do the rest. Normally-open momentary switch. Cable length is not critical (tens of metres is fine); route it away from the solenoid cable. On press the firmware runs the same 5 s release sequence as a recognised palm, without touching the sensor.

BEFORE YOU CONNECT IT

Use a **momentary** (push-and-release) button, not a latching one. A latching switch left “on” would hold the release request — the 5 s cap still protects the solenoid, but the door would re-release every time the software re-armed.

4.6 The front-panel lights

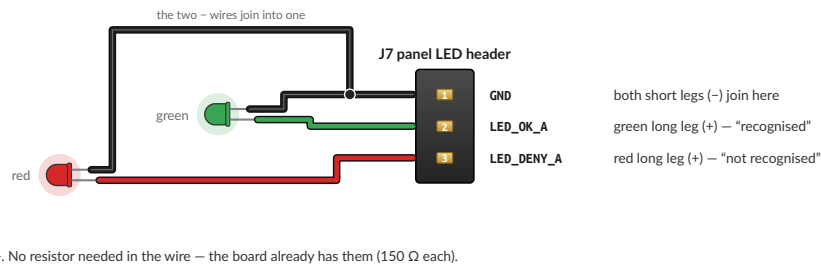


Figure 4.6 — Two ordinary 5 mm LEDs on wires, into the 3-pin header J7. Both short legs share the ground pin; each long leg has its own pin.

Pin	Net	Plain words
1	GND	ground — both LEDs' <i>short</i> legs join here
2	LED_OK_A	green LED's <i>long</i> leg: lights when a palm is recognised
3	LED_DENY_A	red LED's <i>long</i> leg: lights when it isn't

IN PLAIN WORDS

An LED only works one way round: the longer leg is the + side. The board already contains the small resistor each LED needs to limit its current, so the wires are just wires — no extra parts. The screen shows the same information in words; the lights exist so the result can be seen from across the room.

FOR THE ENGINEER

IO1 → R9 150 Ω → J7.2 (green), IO2 → R10 150 Ω → J7.3 (red); ~8 mA each with a 2 V LED from 3.3 V. Any 5 mm LED with $V_f \leq 2.4$ V works; a blue or white LED ($V_f \approx 3$ V) would be dim. Header is a plain 1×3 2.54 mm pin header; use a 3-way Dupont housing so it cannot be plugged in reversed. Pin 1 is square. The yellow LED D4 on the board itself is a separate 3.3 V “power present” indicator (R7 1 kΩ).

BEFORE YOU CONNECT IT

Long leg to pin 2 or 3, short leg to pin 1. Reversed, the LED simply stays dark — nothing is damaged, so if a light never comes on, this is the first thing to check.

4.7 USB-C — programming and logs

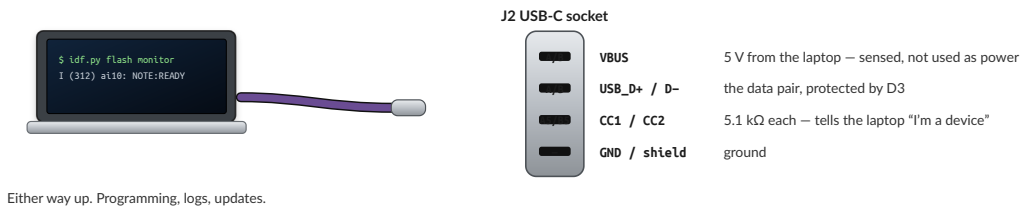


Figure 4.7 — A standard USB-C cable to a laptop, into J2. Either way up. Used only by whoever maintains the unit.

USB-C pins	Net	Plain words
A4 A9 B4 B9	VBUS	the laptop's 5 V. The board only <i>looks</i> at it to know a cable is present; it never runs on it
A6 B6 / A7 B7	USB_D+ / USB_D-	the data pair, protected against static by D3
A5 / B5	CC1 / CC2	two small resistors that tell the laptop "I am a device, not a charger"
A1 A12 B1 B12, shell	GND	ground and the cable's metal shield

IN PLAIN WORDS

The USB port is the back door: it is how the software gets onto the brain and how its diary (the log) gets read out. It does not power the unit — the bolt needs 12 V and a USB port can't supply that — so the adapter must be plugged in even while programming. The board notices when a USB cable is connected and can show it on the screen.

FOR THE ENGINEER

ESP32-S3 native USB on IO19 (D-) / IO20 (D+) — no bridge chip; the S3 enumerates as a CDC serial port and supports DFU. D3 USBLC6-2SC6 ESD array across D+/D- with VBUS reference. R3/R4 5.1 kΩ CC pull-downs (UFP/sink) so the host applies VBUS in either orientation. No VBUS power path by design (no diode-OR, no 5 V → 3.3 V regulator from USB): 12 V is required for U2/U3 anyway. R11 100 kΩ / R12 150 kΩ divide VBUS to 3.0 V on IO7 (USB_DET) so firmware can hold off the D+ pull-up until a host is present. Shield is tied straight to GND (prototype-appropriate; an RC to chassis is an EMC refinement). Recovery if USB enumeration ever breaks: hold BOOT (S2), tap RESET (S1), or use J6.

BEFORE YOU CONNECT IT

Plug the **12 V adapter in first**, then USB. With only USB connected nothing lights up — that is expected, not a fault.

4.8 The debug header (normally empty)

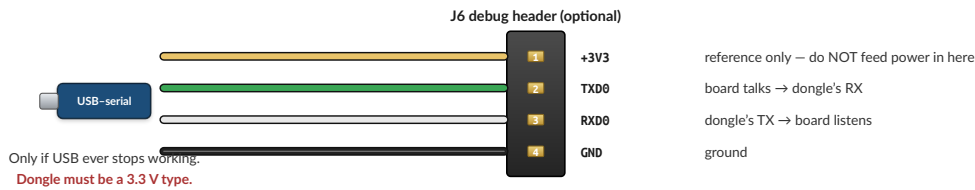


Figure 4.8 — Four pins for a USB-to-serial dongle. A recovery path only; in normal life nothing is connected here.

Pin	Net	Plain words
1	+3V3	3.3 V <i>out of</i> the board — a reference for the dongle, never a way to power the board
2	TXD0	board talks → connect to the dongle's RX
3	RXD0	board listens ← connect to the dongle's TX
4	GND	ground

IN PLAIN WORDS

This is the same kind of two-wire conversation as the sensor cable, but between the brain and a laptop. It exists for one reason: if a software mistake ever leaves the USB port not responding, this header still works, because it is wired to the brain's original console. Notice the crossing: the board's "talk" pin goes to the dongle's "listen" pin and vice-versa — the most common mistake in the world of serial cables.

FOR THE ENGINEER

UART0 console: IO43 (U0TXD) → J6.2, IO44 (U0RXD) ← J6.3. 1×4 2.54 mm pin header. Works with any 3.3 V USB–UART bridge (CP2102, CH340 set to 3.3 V, FT232 at 3.3 V). ROM bootloader is reachable via this port with BOOT held at reset, so a wiped or broken USB stack is recoverable. Pin 1 supplies +3V3 for level reference only — do not back-feed the rail from a dongle.

BEFORE YOU CONNECT IT

The dongle must be a **3.3 V** type. A 5 V one can damage the brain's pins. And never connect the dongle's 5 V or 3.3 V output to pin 1 — pin 1 is an output.

5 Inside the board — the parts and what they do

Forty-six parts sit on the board. Most of them exist to protect something, store a little energy, or keep a signal clean. Here is each one in ordinary words, with the engineering detail beside it.

Parts are named as printed on the board: **U** chips and modules, **D** diodes and LEDs, **C** capacitors, **R** resistors, **Q** transistors, **F** fuses, **J** / **CN** connectors, **S** switches, **BZ** buzzer.

FOR THE ENGINEER

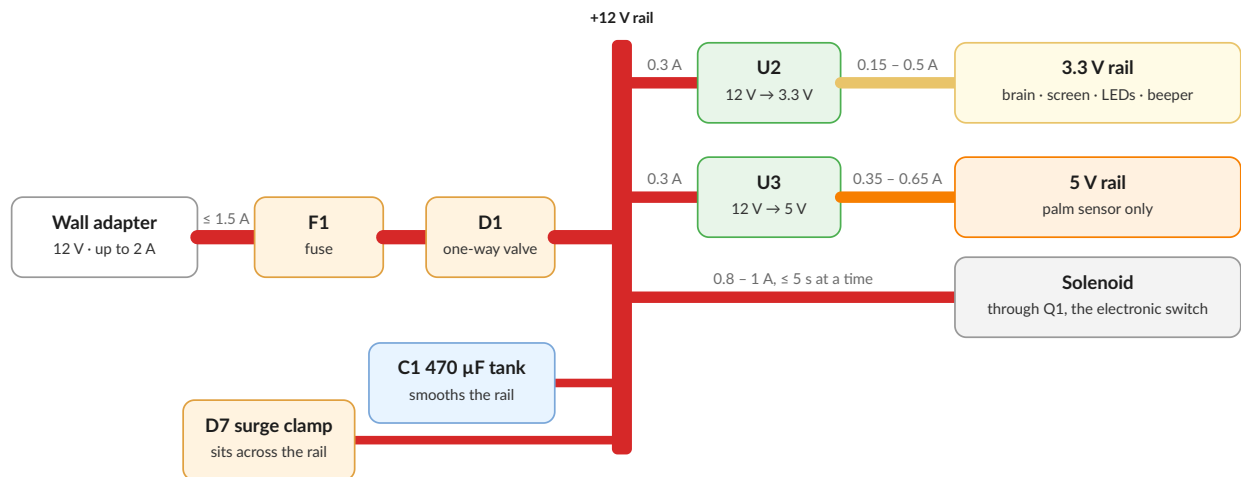
Parts count by category: 1 module, 2 regulators, 1 fuse, 4 diodes (D1, D2, D7, plus the D3 array), 1 LED, 1 MOSFET, 12 capacitors, 13 resistors, 2 switches, 1 buzzer, 9 connectors — 46 on-board, plus the two panel LEDs. Full BOM with sourcing notes in `docs/COMPONENTS.md`; every value is justified in `docs/POWER_BUDGET.md` or `docs/DESIGN_REVIEW.md`.

Ref		Part	In plain words	For the engineer
U1		ESP32-S3-WROOM-1 (N8R2)	The brain. A complete small computer on one module, with Wi-Fi, Bluetooth and USB built in, plus its own antenna printed on the top edge.	Dual-core Xtensa LX7 at 240 MHz, 8 MB flash, 2 MB PSRAM, native USB. Antenna keep-out on the board edge; 3 GND pads incl. thermal pad. Strapping pins IO0/IO3/IO45/IO46 left free except IO0 (BOOT).
U2		RECOM R-78E3.3-1.0	Pressure reducer, 12 → 3.3 V. Makes the gentle voltage the brain, screen and lights run on. Runs cool.	Switching regulator, 1 A, ~93 % efficient, 3-pin SIP, needs ≥ 7 V in. Loss ≈ 0.09 W. Drop-in alternative K7803-1000R3.
U3		RECOM R-78E5.0-1.0	Pressure reducer, 12 → 5 V. Feeds only the sensor.	Switching regulator, 1 A. Must be the 1.0 A part (same body as 0.5 A). ≈ 0.35 W loss at worst load; copper pour suffices.
F1		Polyfuse 2 A hold / 16 V	Automatic circuit breaker. If something shorts, it goes high-resistance and cuts the current; when it cools, it resets itself.	Bourns MF-MSMF200/16, 1812. 4 A trip. Guards against an oversized adapter; with a 2 A SMPS adapter the adapter's own limit acts first.
D1		SS34 Schottky diode	One-way valve. Lets power in only if the adapter is the right way round. Backwards, nothing flows and nothing breaks.	3 A, 40 V, ~0.5 V forward drop (≈ 0.5 W during a 5 s release). Everything on the board sits behind it.
D7		SMAJ15A TVS	Surge clamp. Swallows short voltage spikes — from the lock cable acting as an antenna, or a noisy adapter — before they reach anything delicate.	15 V stand-off, ~24 V clamp, 400 W peak. Protects Q1 (30 V) and the regulators.
C1		470 µF 25 V electrolytic	The big tank. Stores a little energy so that when the bolt suddenly draws current, the voltage doesn't sag.	Low-ESR 105 °C, 8 mm radial. Also absorbs the switching ripple from U2/U3. C5/C7 10 µF at the regulator inputs.
C2, C12		100 µF 10 V, ×2	Smaller tanks at the 3.3 V output and right at the sensor socket, for the sensor's start-up gulp.	SMD electrolytic/polymer 6.3 × 5.4 mm (a 100 µF MLCC in 0805 does not exist). Within RECOM's 220 µF output-capacitance limit.

Ref		Part	In plain words	For the engineer
C3, C6, C8, C4, C10, C11		100 nF ... 10 μ F ceramics	Tiny tanks placed right next to each chip's power pin, so fast demands are met locally.	C3 100 nF at U1 pin 2 (≤ 5 mm), C6/C8 10 μ F at regulator outputs, C4 1 μ F on EN (10 ms reset RC with R1), C10/C11 100 nF button debounce.
Q1		AO3400A N-MOSFET	Electronic switch for the bolt — like a relay with no moving parts. The brain nudges it with 3.3 V; it switches the full 12 V current.	SOT-23, 30 V, 5.7 A, $R_{DS(on)} \approx 40$ m Ω , $V_{GS(th)} \leq 1.45$ V. Low-side. R13 100 Ω gate series, R2 10 k Ω gate pull-down.
D2		SS54 Schottky diode	Kick-back catcher. When the switch opens, the coil tries to keep current flowing; this diode gives it a safe path instead of a damaging spike.	5 A, 40 V, across J3 (cathode to +12V). Flyback / freewheeling diode.
R2		10 k Ω	Safety resistor that holds the switch firmly off while the brain is waking up, so the bolt can never twitch at power-on.	Gate-to-source pull-down; GPIOs are high-impedance for ~ 50 ms after reset.
D3		USBLC6-2SC6	Static guard on the USB data wires. Your finger on the plug can carry thousands of volts of static; this shunts it away.	Ultra-low-capacitance ESD array, SOT-23-6, on D+/D- with VBUS reference.
R3, R4		5.1 k $\Omega \times 2$	Tell the laptop “I’m a device” so it switches on its USB power for either plug orientation.	CC1/CC2 pull-downs, USB-C UFP sink.
R11, R12		100 k Ω / 150 k Ω	USB detector. Lets the brain see that a cable is plugged in.	VBUS divider $\rightarrow$ 3.0 V on IO7 (USB_DET).
BZ1, R5		Passive piezo, 220 Ω	The beeper. One beep for welcome, two for denied.	Passive piezo element driven by a PWM tone on IO5 through 220 Ω (≈ 15 mA edge current). A magnetic buzzer would need a transistor — don't substitute.
S1, S2		Tactile buttons	Reset restarts the brain. Boot , held while pressing Reset, puts it into programming mode. Both only for the maintainer.	S1 $\rightarrow$ EN (with R1 10 k Ω / C4 1 μ F). S2 $\rightarrow$ IO0 strapping with C10 100 nF; internal pull-up.
D4, R7		Yellow LED, 1 k Ω	Power light on the board itself: proves the 3.3 V supply is alive even before the screen wakes up.	0603 LED on +3V3, ~ 1.3 mA.
R6, R8, C11		10 k Ω , 1 k Ω , 100 nF	Make the exit button read cleanly and protect the brain from the long wire.	Pull-up, series guard, RC debounce on IO6 (Section 4.5).
R9, R10		150 $\Omega \times 2$	Current limiters for the two panel lights .	~ 8 mA each from 3.3 V GPIOs (Section 4.6).

6 Where the power goes

One 12 V supply comes in. Three voltages come out of it. Each part gets the voltage it was built for.



Everything right of D1 is protected: reversed adapter → nothing flows; short on the board → F1 opens, then resets when cool.

At the adapter: ≈ 0.25 A idle, ≈ 1.2 A while the bolt is pulled — well inside a 2 A adapter.

Figure 6.1 — The power tree. Left to right: adapter, fuse, one-way valve, then the +12 V rail. Two regulators step it down; the solenoid takes it straight. Widths are roughly proportional to current.

IN PLAIN WORDS

Think of the 12 V rail as the mains water pipe in a house. The fuse is a stopcock that shuts itself if a pipe bursts; the one-way valve stops water flowing back into the street. The big tank (C1) keeps the pressure steady when a tap opens suddenly — that tap is the solenoid, the thirstiest thing in the box. Two pressure reducers (U2, U3) feed the delicate things: the brain and screen at 3.3 V, the sensor at 5 V.

Rail	Made by	Feeds	Typical	Worst
+12V	adapter via F1, D1	U2, U3, solenoid (via Q1), C1, D7	0.25 A idle	1.5 A for ≤ 5 s
+5V	U3 (1 A)	sensor only	0.35 – 0.43 A	0.63 A + start-up spike
+3V3	U2 (1 A)	brain, screen + backlight, panel LEDs, buzzer, pull-ups, debug header	0.15 A	0.52 A (Wi-Fi burst)
VBUS	laptop	nothing — sensed only (R11/R12 → IO7)	0	0

FOR THE ENGINEER

Worst-case dissipation: D1 0.53 W (5 s bursts), U3 0.35 W, U2 0.09 W, F1 0.1 W, Q1 0.03 W — copper pours suffice, no heatsinks. Trace widths (POWER_BUDGET §6): +12V trunk to J3 and +5V trunk to CN1 ≥ 1.0 mm at 1 oz, +12V to U2/U3 0.6 mm, signals 0.2–0.25 mm. A 3 A adapter is acceptable and lets F1 (4 A trip) act on a hard short before the adapter does. Unknowns: AI10 inrush (mitigated by C12), display current (est. 100 mA).

7 What happens during a scan

From switching on to a door opening, in seven steps — and which wire each step travels on.

One scan, start to finish

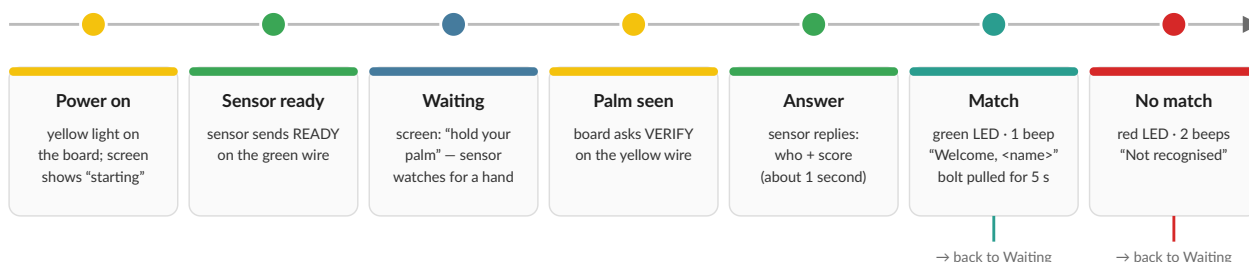


Figure 7.1 — One scan. Yellow steps are messages from the board to the sensor (yellow wire); green steps are the sensor's replies (green wire).

- Power on.** The adapter is plugged in. The yellow light on the board comes on within a fraction of a second (the 3.3 V rail is alive). The brain boots and shows "starting" on the screen. The sensor, on its own 5 V, boots too.
- Sensor ready.** A few seconds later the sensor sends a "ready" message down the green wire. Until it does, the board sends nothing — asking too early is ignored or answered with an error.
- Waiting.** The screen says "hold your palm over the sensor". The sensor watches for a hand at the right distance; the board listens.
- Palm seen.** The board sends the "verify" command on the yellow wire. The sensor lights its infrared LEDs, photographs the veins, and compares the pattern with the people it has stored.
- Answer.** About a second later the reply arrives on the green wire: either a person number and a confidence score, or "nobody".
- Match.** The board switches the green light on (J7 pin 2), beeps once (BZ1), shows "Welcome" on the screen, and switches Q1 on so 12 V flows through the bolt for five seconds. Then Q1 opens, D2 catches the coil's kick, the spring pushes the bolt out, and everything returns to *Waiting*.
- No match.** Red light (J7 pin 3), two low beeps, "Not recognised". Back to *Waiting*.

IN PLAIN WORDS

The exit button skips steps 3–5 entirely: pressing it produces the same five-second release. Adding a person is the same conversation with a different question — the screen guides the maintainer through it, and the sensor stores the new pattern in its own memory, not on the board.

FOR THE ENGINEER

Firmware skeleton: UART1 task (framing, XOR check, NOTE:READY gate) → event queue → application state machine (IDLE → VERIFY_PENDING → RELEASE(5 s) → IDLE) → outputs. The release timer is a hardware timer, and the ESP-IDF task watchdog is armed on the application task; a hung task resets the chip, and R2 guarantees Q1 is off through the reset. VERIFY (0x12) carries a timeout byte; NOTE messages report face/palm position while the module is searching, so the screen can say "a little higher". Score threshold and template management stay inside the AI10.

8 Before you power it on

Ten minutes with a multimeter, in this order. Each line prevents a specific way of breaking something.

BENCH, BEFORE ANYTHING IS PLUGGED INTO THE BOARD

- ☐ Adapter measures **11.5 – 12.6 V** with nothing connected, and its label shows $\oplus$ on the centre pin.
- ☐ Sensor, powered on its own: the **green wire idles at ≤ 3.6 V**. If it reads 5 V, stop and ask for the divider.
- ☐ Solenoid coil resistance is roughly **10 – 20 Ω** ($\approx 0.6 – 1.2$ A at 12 V). An open circuit or under 5 Ω means a faulty unit.
- ☐ Screen module's pin order matches the table in Section 4.3, read off its silkscreen.
- ☐ Buzzer says **passive** and **piezo** on the listing.

BOARD, EMPTY (NO MODULES, NO ADAPTER)

- ☐ Resistance from **+12V** to **GND** is not a short (should climb as C1 charges through the meter).
- ☐ No solder bridges across Q1, D3, or the regulator pins.

FIRST POWER

- ☐ Adapter only — nothing else plugged in. The **yellow light** comes on.
- ☐ Measure: **+12V** ≈ 11.5 V (12 V minus D1), **+5V** = 4.9 – 5.1 V, **+3V3** = 3.2 – 3.4 V.
- ☐ Power off. Plug in the screen (pin 1 to pin 1). Power on: backlight lights.
- ☐ Power off. Plug in the sensor. Power on: the sensor's IR LEDs may glow faintly on a phone camera; the board reports "ready" in the log.
- ☐ Power off. Connect the solenoid. Power on, press the exit button: bolt pulls for 5 s and releases.

ALWAYS

- ☐ One power source into the board at a time (J8 or J1).
- ☐ USB is never the power source. 12 V first, USB second.
- ☐ A door secured by this bolt opens from inside by hand.

IF SOMETHING IS WRONG

If anything smells hot, gets hot to the touch, or the yellow light flickers: unplug the adapter first, think second. The fuse resets itself once it cools, so a flickering light usually means a short somewhere downstream.

A Appendix — pin tables

The complete reference for every connector, in one place. Net names are the schematic labels. Pin 1 is marked on the board.

J8 · DC jack 5.5 × 2.1	Net
centre	VIN_12V
sleeve	GND

J1 · 12 V screw terminal	Net
1	VIN_12V
2	GND

J3 · lock terminal	Net
1	+12V
2	LOCK_SW

J5 · exit button	Net
1	EXIT_SW
2	GND

J7 · panel LEDs	Net
1	GND
2	LED_OK_A
3	LED_DENY_A

J4 · display, 14-pin	Net
1 VCC	+3V3
2 GND	GND
3 CS	TFT_CS
4 RESET	TFT_RST
5 DC	TFT_DC
6 SDI	SPI_MOSI
7 SCK	SPI_SCK

J6 · debug UART	Net
1	+3V3
2	TXD0
3	RXD0
4	GND

CN1 · sensor (PicoBlade 1.25 mm)	Net
1 black	GND
2 yellow	SEN_RX
3 green	SEN_TX
4 red	+5V

J2 · USB-C	Net
A1 B1 A12 B12	GND
A4 A9 B4 B9	VBUS
A5	CC1
B5	CC2
A6 B6	USB_D+
A7 B7	USB_D-
A8 B8	SBU – not connected
S1 (shell)	GND

J4 · display, 14-pin	Net
8 LED	+3V3
9 SDO	– (NC)
10 T_CLK	SPI_SCK
11 T_CS	TOUCH_CS
12 T_DIN	SPI_MOSI
13 T_DO	SPI_MISO
14 T_IRQ	TOUCH_IRQ

ESP32-S3 pin use

GPIO	Net	Goes to
EN	EN	reset (S1, R1, C4)
IO0	BOOT	boot strap (S2, C10)
IO1	LED_OK	green panel LED → R9 → J7.2
IO2	LED_DENY	red panel LED → R10 → J7.3
IO4	LOCK_EN	solenoid switch → R13 → Q1 gate
IO5	BUZZER	piezo → R5 → BZ1
IO6	EXIT_BTN	exit button (R6, R8, C11) ← J5.1
IO7	USB_DET	VBUS present (R11/R12)
IO8	TOUCH_IRQ	J4.14
IO9	TOUCH_CS	J4.11
IO10	TFT_CS	J4.3
IO11	SPI_MOSI	J4.6, J4.12
IO12	SPI_SCK	J4.7, J4.10
IO13	SPI_MISO	J4.13
IO14	TFT_DC	J4.5
IO17	SEN_RX	U1TXD → CN1.2 (yellow)
IO18	SEN_TX	U1RXD ← CN1.3 (green)
IO19	USB_D-	J2
IO20	USB_D+	J2
IO21	TFT_RST	J4.4
IO43	TXD0	J6.2
IO44	RXD0	J6.3
IO3, IO45, IO46	–	strapping pins, deliberately unconnected

B Appendix — glossary

Term	Meaning
Capacitor	A tiny rechargeable tank for electricity. Big ones smooth the supply; small ones sit next to chips to answer sudden demands.
Debounce	Smoothing the chatter of a mechanical button so one press is read as one press.
Diode	A one-way valve for current. A Schottky diode is a low-loss kind; a TVS diode is one built to swallow spikes; an LED is one that lights up.
ESD	Electrostatic discharge — the spark from a finger. Thousands of volts, harmless to you, fatal to a chip.
Fail-secure	When power is lost, the lock stays locked. (The opposite, fail-safe, unlocks.)
Flyback diode	The diode across a coil that gives its stored energy somewhere safe to go when the coil is switched off.
Fuse (polyfuse)	A self-resetting fuse: it becomes a high resistance when too much current flows, then recovers when it cools.
MOSFET	An electronic switch. A small voltage on its gate lets a large current flow between its other two pins.
NIR	Near-infrared light, 700–900 nm — just beyond what the eye can see. Blood absorbs it; skin lets it through; that is why veins show up dark.
Pull-up / pull-down	A resistor that gives a wire a definite resting level (high or low) so it doesn't float and pick up noise.
Regulator	A circuit that turns one voltage into another, steadily. Switching regulators do it efficiently and run cool.
Strapping pin	A pin the ESP32 reads at the instant of reset to decide how to start. Left alone unless you mean it.
Watchdog	A timer the software must keep resetting. If the software hangs, the timer expires and restarts the chip.

References

- palm-scan design documents: `docs/NETLIST.md`, `docs/COMPONENTS.md`, `docs/POWER_BUDGET.md`, `docs/DATASHEETS.md`, `docs/DESIGN_REVIEW.md`; schematic `hardware/kicad/palm-scan.pdf`.
- DFRobot, *SEN0677 AI10 wiki* and *User Software Development Manual V1.0* (2025).
- Espressif, *ESP32-S3-WROOM-1 Datasheet* v1.8.
- RECOM, *R-78E series* datasheet; Alpha & Omega, *AO3400A*; Vishay, *SS34*; ST, *USBLC6-2SC6*.
- W. Wu et al., “Review of palm vein recognition,” *IET Biometrics* 9(1), 2020.

Generated from the Rev 2.4 netlist by `docs/wiring-guide/build.py`. The board drawing in Section 3 is illustrative; connector positions are fixed during PCB layout. Makers: Ashmita K Rao, Adithya Satish, Vinod Kumar.